MOCO: Measure Once, Compile Once

A noise-invariant variational classifier for credit-card fraud, trained with two circuit executions per example

Darius Tatomir Deliu Georgiana

BreaQ 2026, Quantum AI track, Task 2 “Fraud on a Quantum Budget”

Abstract

Training a variational quantum classifier (VQC) on hardware is expensive: every gradient step costs circuit executions, and depolarizing noise both compresses the scores and reorders them. We present MOCO (Measure Once, Compile Once), a four-qubit product circuit, RY encoding followed by one trainable rotation per qubit and a weighted Z readout. In the Heisenberg picture the readout is linear in the Bloch vectors of the encoded data, so training reduces to a convex logistic regression on data measured once (two executions per training transaction); the optimum is compiled back into rotation angles and observable weights. Because every qubit sees the same number of gates and no entangling gate, depolarizing noise multiplies every score's distance to 1/2 by the same factor λ^2, $\lambda = 1 - 4p/3$: AUPRC, AUROC and the ranking are identical at every noise level, and the decision threshold moves analytically. Under the organizers' protocol (common split, 5 seeds, noise after every native gate), MOCO reaches a test AUPRC of 0.716 ± 0.007 at every p, while the kit baseline falls from 0.580 ± 0.053 to 0.405 ± 0.177 at $p = 0.05$; a logistic regression on the same inputs reaches 0.710 ± 0.007, so MOCO matches, and does not beat, the classical reference. Training and model selection cost 6,490 executions instead of 2,859,360. With finite shots the noise becomes a shot tax of λ^{-4}, which an anytime-valid adaptive triage reduces to 208.5 shots per transaction at $p = 0.05$ (1024 fixed) at the same F1. The circuit is classically simulable and we claim no quantum advantage.

1 Introduction

Variational quantum algorithms [1] train parametrized circuits with a classical optimizer. On hardware, each gradient is estimated from circuit executions: with the parameter-shift rule [2], a circuit with P trainable angles costs $1 + 2P$ executions per example and per epoch. The kit's baseline VQC for this task, which follows the pipeline of Dinuţ et al. [3], is a ZZ feature map followed by an EfficientSU2-style ansatz with 32 parameters trained for 30 epochs on a balanced set of 590 rows; by the kit's formula it needs 1,150,500 training executions and 1,708,860 validation executions. Noise adds a second problem. Depolarizing channels after each of its 126 noisy gate locations compress the scores towards 1/2 non-uniformly, so the ranking of transactions changes as well [3, 4].

Fraud detection makes the ranking the quantity of interest: the ULB credit-card dataset has 492 frauds among 284,807 transactions [5], and the protocol ranks methods by the area under the precision-recall curve (AUPRC). A noise model that only compresses the scores uniformly would leave AUPRC unchanged; one that reorders them cannot be repaired by any number of shots.

Contributions.

1. **MOCO**, a product circuit whose training is an exact convex reparametrization of the VQC loss: the training data are measured once (two executions per row, 6,490 in total) and the global optimum is compiled back into the circuit, with no circuit inside the optimization loop.
2. A **rank-invariance theorem** under the official noise model, with an analytic threshold transfer and a shot tax of exactly λ^{-4} in signal compression, checked against the kit's density-matrix simulator.
3. An evaluation under the **official protocol** (5 seeds), a cross-validation on all 492 frauds, a cost ledger under three definitions, ablations and a shallow entangled warm-start variant.
4. **Adaptive shot triage** with anytime-valid confidence intervals and a **noise meter** that estimates p from 1,000 calibration executions, so that the threshold needs no noisy validation pass.

We state the limits plainly: MOCO is classically simulable and, as a model class, a logistic regression on the same angles; its contribution is robustness, stability and cost, not accuracy beyond the classical reference.

2 Setup

Data and split. The dataset has 30 columns: 28 anonymized principal components V1–V28, Time and Amount [5]. The common split is stratified 60/20/20

with seed 42, before any balancing: 170,883 training rows (295 frauds), 56,962 validation rows (99 frauds) and 56,962 test rows (98 frauds). The test fraud rate, i.e. the AUPRC of a random ranking, is 0.172%.

Official protocol. The organizers' evaluation, which we reproduce cell by cell in `qai/protocol.py`, decomposes every circuit into $\{RX, RY, RZ, \mathrm{Rot}, H, \mathrm{CNOT}, \mathrm{CZ}\}$ and inserts a depolarizing channel of probability p after every gate, on each qubit it touches, simulated exactly with density matrices, for $p \in \{0, 0.001, 0.01, 0.05\}$. The threshold maximizes F1 on the validation scores computed at the same p. We report test AUPRC, AUROC, the Spearman correlation ρ between noisy and noiseless test scores, F1 and MCC, as mean ± standard deviation over seeds 0–4. Balancing, encoding, architecture, epochs, hyperparameters and thresholds are chosen on training or validation data only; **the test set is used only for reporting**.

Encoding. As in the kit, Time and Amount are standardized, the 30 columns are projected onto four principal components, and each component is scaled to an angle in $[0, \pi]$; all three maps are fitted on the undersampled training subset only.

Kit baseline and its ledger. The baseline is the VQC of Task 1: 93 native gates, 33 CNOTs, depth 45 and 126 noise locations. Its kit-formula ledger is 2,859,360 executions for training and validation; counted with `qml.Tracker`, a training step costs 41 executions per example instead of 65, because PennyLane does not shift parameters that cannot reach the measured qubit, which gives 2,434,560. Removing the 15 gates outside the light cone of the readout leaves the scores unchanged (unit test in `tests/test_protocol.py`), so pruning saves neither noise nor executions.

3 Method

Circuit. MOCO uses four qubits. Qubit q receives $RY(x_q)$, which loads angle x_q, followed by a trainable $\mathrm{Rot}(\phi_q, \theta_q, 0)$; the readout is $H = \sum_q w_q Z_q$ with $w_q \geq 0$, $\sum_q w_q = 1$, and the fraud score is $s(x) = (1 - \langle H \rangle)/2$. The circuit has 8 native gates, no CNOT, depth 2 and 8 noise locations.

Heisenberg picture. $RY(x)|0\rangle$ has Bloch vector $r(x) = (\sin x, 0, \cos x)$. A single-qubit rotation U acts on Bloch vectors as a rotation $R_U \in SO(3)$, so $\langle Z_q \rangle = \hat{z} \cdot R_U r(x_q) = m_q \cdot r(x_q)$ with $m_q = R_U^\top \hat{z}$ a unit vector: the trainable rotation only turns the measurement axis. Hence

$$\langle H \rangle = \sum_q w_q \, m_q \cdot r(x_q) = \sum_q \big(c_q^x \sin x_q + c_q^z \cos x_q \big), \quad (1)$$

with $c_q = w_q(m_q^x, m_q^z)$, is linear in the measured features $(\sin x_q, \cos x_q)$.

Measure once, fit convexly. On hardware, the Bloch components of all four qubits are read in two settings (X and Z bases), so the training set costs two executions per row, 6,490 for the 3,245 rows of the 1:10 undersampled set. Training the circuit with a logistic loss on (1) plus an intercept is an ℓ_2-regularized logistic regression on these 8 measured features: strictly convex, solved to its global optimum with L-BFGS in PyTorch, with the regularization chosen by 5-fold cross-validation inside the training set, so that no validation executions are spent and no circuit runs inside the loop.

Compile. Let $\beta_q \in \mathbb{R}^2$ be the fitted coefficients of qubit q on $(\sin x_q, \cos x_q)$. Setting $m_q = -\beta_q / \|\beta_q\|$ (realized by $\mathrm{Rot}(-\phi_q, -\theta_q, 0)$, with (θ_q, ϕ_q) the polar angles of m_q) and $w_q = \|\beta_q\| / \sum_{q'} \|\beta_{q'}\|$ gives $s(x) = \frac{1}{2} + \sum_q \beta_q \cdot (\sin x_q, \cos x_q)/(2 \sum_{q'} \|\beta_{q'}\|)$, a strictly increasing affine function of the fitted logit without intercept. The compiled circuit therefore ranks transactions exactly as the convex model, and the intercept is absorbed by the threshold. The circuit, not a classical post-processor, performs the classification.

Choices made on validation. The undersampling ratio and the feature map were chosen on validation AUPRC over 5 seeds before any test evaluation: single 1:10 reached 0.675 ± 0.012 with 6,490 training executions; pair 1:30 tied (0.675 ± 0.010) at 27,435 executions, and the cheaper configuration was kept.

4 Theory

Theorem 1 (Rank invariance)**.** *Let a circuit on n qubits consist only of single-qubit gates, with exactly k native gates on every qubit, followed by measurement of $H = \sum_q w_q Z_q$. Under the official noise model, a depolarizing channel $\mathcal{D}_p(\rho) = (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z)$ after every gate, for every input x*

$$s_p(x) = \tfrac{1}{2} - \lambda^k \big(\tfrac{1}{2} - s_0(x) \big), \qquad \lambda = 1 - \tfrac{4p}{3}. \quad (2)$$

Proof. A single-qubit state is $\rho = \frac{1}{2}(I + r \cdot \sigma)$ with Bloch vector r, $\|r\| \leq 1$. *(i) Channel.* Since $X\sigma_j X = \pm\sigma_j$ with a minus sign exactly for $j \neq x$, and similarly for Y and Z, each Pauli component σ_j is kept by one of the three Pauli conjugations and flipped by the other two. Hence $\mathcal{D}_p$ maps $\sigma_j \mapsto (1-p)\sigma_j + \frac{p}{3}(1-2)\sigma_j = (1 - \frac{4p}{3})\sigma_j$ and leaves I fixed: $r \mapsto \lambda r$. *(ii) Gates.* A unitary U maps

$r \mapsto R_U r$ with $R_U \in SO(3)$. Since λr is a scalar multiple, $R_U(\lambda r) = \lambda R_U r$: the channel commutes with every single-qubit gate. *(iii) Product structure.* Without multi-qubit gates and with product input $|0\rangle^{\otimes n}$, every gate and every channel acts on one qubit, so the state remains a product $\bigotimes_q \rho_q$ and each qubit evolves independently. After its k gates, each followed by a channel, qubit q has Bloch vector $\lambda^k R_{U_k} \cdots R_{U_1} r_{q,0} = \lambda^k r_q^{\text{ideal}}$ by (i) and (ii). *(iv) Readout.* $\langle Z_q \rangle_p = \hat{z} \cdot \lambda^k r_q^{\text{ideal}} = \lambda^k \langle Z_q \rangle_0$, so $\langle H \rangle_p = \lambda^k \langle H \rangle_0$ by linearity and $s_p = (1 - \lambda^k \langle H \rangle_0)/2$, which is (2). □

For MOCO, $k = 2$ (RY and Rot are both native). Since $\lambda^2 > 0$ for every $p \neq 3/4$, the map $s_0 \mapsto s_p$ is strictly increasing, which gives:

Corollary 1 (Metrics and threshold)**.** *For every $p \in [0, 3/4)$: (a) AUPRC and AUROC of s_p equal those of s_0, and the Spearman correlation between s_p and s_0 is 1; (b) the classifications $\{s_p \geq g_p(t)\}$ and $\{s_0 \geq t\}$ coincide with $g_p(t) = \frac{1}{2} - \lambda^2(\frac{1}{2} - t)$, so the threshold maximizing F1 on noisy validation scores is $t_p = g_p(t_0)$, and precision, recall, F1 and MCC at the transferred threshold are identical to those at $p = 0$.*

Part (a) needs only that ranking metrics are invariant under strictly increasing maps; part (b) holds because g_p is a bijection of the score range. The threshold t_0 can be computed classically (the noiseless scores of a product circuit are closed-form), so MOCO needs no noisy validation pass once p is known.

Numerical check. Against the kit's density-matrix simulator (`noisy_predict`), the contraction (2) holds to 5.7×10^{-14} on 200 rows at $p = 0.05$ and $p = 0.5$, the joint 16-outcome distribution equals the product of the per-qubit marginals to 8.6×10^{-14}, and the measured median compression equals λ^2 to 2.8×10^{-17} at every tested p. In the adaptive-shots run, the transferred threshold matches the threshold chosen on the noisy validation set to 10^{-9} or better.

Shot tax. On hardware each score is estimated from S shots; the per-shot estimator $h = \sum_q w_q z_q \in [-1, 1]$ has variance at most 1, so the standard error of $\hat{s}$ is at most $1/(2\sqrt{S})$ at every p. By (2), the difference between the scores of two transactions shrinks by λ^2. Keeping the same signal-to-noise ratio as without noise therefore requires $S_p/S_0 = \lambda^{-4}$ under this bound, i.e. 1.32× at $p = 0.05$, 3.46× at $p = 0.2$ and 81× at $p = 0.5$. We define the shot tax of any model as $1/c^2$, with c its measured median compression median $|s_p - \frac{1}{2}|/|s_0 - \frac{1}{2}|$; for MOCO $c = \lambda^2$ exactly. Because the exact per-shot variance also grows slightly with p, the tax is a lower bound on the shots needed.

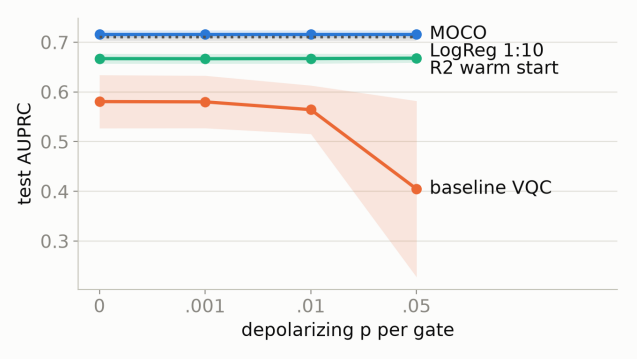


Figure 1: Test AUPRC under the official protocol (5 seeds). Source: `a_auprc_protocol.png`, from the per-seed CSVs of Table 1.

Link to Task 1. The theorem combines two exercises of the Task 1 notebook. Exercise 3.2 shrinks the scores of a classifier towards 0.5 and observes that AUPRC is unchanged while the recall at a fixed threshold changes; exercise 10.1 checks that a depolarizing channel after $RY(\theta)$ gives $\langle Z \rangle = (1 - \frac{4p}{3}) \cos\theta$. MOCO is the circuit for which the second statement implies the first for the whole model.

5 Experiments

All experiments use seeds 0–4 and the official protocol unless stated otherwise; code and per-seed results are in the repository (`results/task2/`).

5.1 Official protocol

Table 1 reproduces the task statement for the reference models: the baseline reaches 0.580±0.053 and the logistic regression on its inputs 0.703±0.010 [3]. At $p = 0.05$ the baseline loses 30% of its AUPRC, its spread over seeds triples and its ranking decorrelates ($\rho = 0.686 \pm 0.145$); its max-F1 threshold collapses to 0.500 ± 0.001. MOCO keeps 0.716 ± 0.007, $\rho = 1$ and F1 0.800 ± 0.019 at every noise level, as Corollary 1 predicts, and matches the logistic regression on the same inputs (0.710 ± 0.007). Figure 1 shows AUPRC against p.

5.2 Cost ledger

Table 2 counts circuit executions on hardware (1,024 shots each) for training, model selection and threshold calibration. We report three definitions, from the kit's to the most conservative:

1. *Kit definition* (training and model selection): baseline 2,859,360 against MOCO 6,490, a reduction of 99.77%.
2. *With the noise meter* (Section 5.8): baseline 2,916,322 against MOCO 6,490 + 1,000 = 7,490, −99.74%.

Table 1: Test AUPRC under the official protocol (mean ± std over 5 seeds), Spearman ρ between noisy and noiseless test scores and F1 at $p = 0.05$. LogReg = logistic regression on the same inputs as the model named.

Model	$p = 0$	$p = 0.001$	$p = 0.01$	$p = 0.05$	ρ ($p = 0.05$)	F1 ($p = 0.05$)
Kit baseline VQC (93 gates, 33 CNOT)	0.580 ± 0.053	0.580 ± 0.053	0.564 ± 0.049	0.405 ± 0.177	0.686 ± 0.145	0.537 ± 0.184
R2 entangled, random init (15 gates)	0.635 ± 0.018	0.635 ± 0.018	0.636 ± 0.017	0.638 ± 0.015	0.999 ± 0.001	0.760 ± 0.021
R2 entangled, MOCO warm start	0.667 ± 0.010	0.667 ± 0.010	0.667 ± 0.010	0.668 ± 0.009	0.999 ± 0.001	0.798 ± 0.009
MOCO (8 gates, 0 CNOT)	$\mathbf{0.716 \pm 0.007}$	$\mathbf{0.716 \pm 0.007}$	$\mathbf{0.716 \pm 0.007}$	$\mathbf{0.716 \pm 0.007}$	**1.000**	0.800 ± 0.019
LogReg 1:10 (inputs of MOCO)	0.710 ± 0.007	=	=	=	–	0.794 ± 0.023
LogReg 1:1 (inputs of the baseline)	0.703 ± 0.010	=	=	=	–	0.791 ± 0.019
MOCO, robust encoding $q = 0.5\%$ (not selected)	0.738 ± 0.006	0.738 ± 0.006	0.738 ± 0.006	0.738 ± 0.006	1.000	0.783 ± 0.024
LogReg 1:10, same robust inputs	0.731 ± 0.004	=	=	=	–	0.785 ± 0.020

Table 2: Cost ledger, circuit executions (formula; Tracker count in parentheses where it differs). Training + model selection + threshold calibration.

Method	Train	Select	Calib.	Total
Baseline (65/ex.)	1,150,500	1,708,860	56,962	2,916,322
Baseline (Tracker, 41/ex.)	725,700	1,708,860	56,962	2,491,522
R2 warm start (49/ex.)	298,835	113,920	56,962	469,717
MOCO (2/row)	**6,490**	**0**	56,962	**63,452**
MOCO, noise meter	6,490	0	1,000	7,490

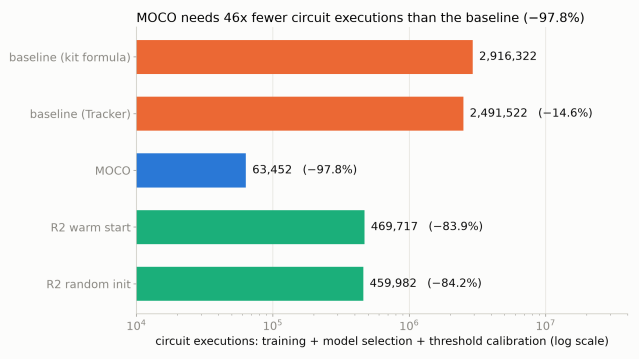


Figure 2: Cost ledger (log scale). Source: `c_ledger.png`, from `ledger_final.csv`.

3. *Most conservative*, threshold calibrated on the full noisy validation set (56,962 executions): MOCO 63,452, −97.82% (formula) and −97.45% against the Tracker count of the baseline.

5.3 Finite shots and the shot tax

The protocol evaluates exact expectation values. With $S = 1024$ shots per transaction at $p = 0.05$, test AUPRC is 0.705 for MOCO, 0.643 for R2 with warm start and 0.007 for the baseline, which falls to chance level (Figure 3). The measured compression at $p = 0.05$ is $0.871 = \lambda^2$ for MOCO, 0.795 for R2 and 0.045 for the baseline, i.e. shot taxes of 1.32×, 1.58× and at least 504× (between 250× and 2.2×10^5× over seeds). For the baseline the tax is only a lower bound: its compression is not uniform and its ranking is lost even with infinite shots.

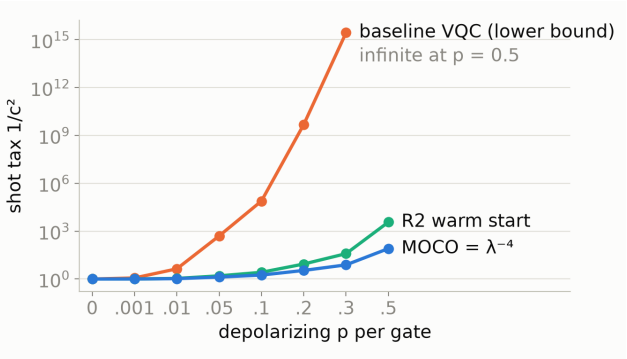


Figure 3: Shot tax $1/c^2$ against p (diagnostic, measured on test, not used for any choice). Source: `e_shot_tax.png`, from `shot_tax.csv`.

5.4 All 492 frauds and the AUPRC/ρ paradox

The official test set contains only 98 frauds. We therefore ran a stratified 5-fold cross-validation of the whole dataset in which every fraud is tested once (Figure 4). MOCO reaches 0.721 ± 0.020 at every p and the logistic regression on the same inputs 0.739 ± 0.030; the paired difference per fold is -0.018 ± 0.026, so the two cannot be distinguished, and the small advantage of MOCO on the official test set does not persist. The baseline reaches 0.499 ± 0.137, with a spread over folds about seven times that of MOCO.

The same folds show the paradox reported by Dinuţ et al. [3]: at low noise the baseline's AUPRC rises in 5 of 5 folds at $p = 0.001$ and $p = 0.01$ (mean 0.529 ± 0.118 at $p = 0.01$), while its ranking decays ($\rho = 0.931 \pm 0.122$ at $p = 0.01$, 0.669 ± 0.306 at $p = 0.05$). A single metric can thus report an improvement where the model gets worse. Its drop at $p = 0.05$ appears in only 3 of 5 folds. MOCO cannot show the paradox, because its ranking is invariant.

5.5 Shallow entanglement with warm start (R2)

R2 inserts three CNOTs between the encoding rotations and MOCO's trainable layer: $RY(x) \to \mathrm{Rot}(\alpha) \to$

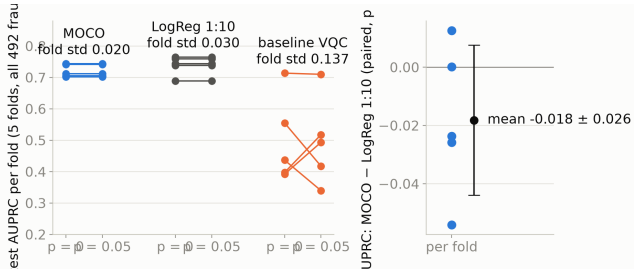


Figure 4: Five-fold cross-validation on all 492 frauds. Source: `i_cv_all_frauds.png`, from `cv/cv_per_fold.csv`.

Table 3: Feature-map ablations with the same convex training (test AUPRC, 5 seeds).

Map	CNOT	$p = 0$	$p = 0.05$	ρ ($p = 0.05$)
single	0	0.716 ± 0.007	0.716 ± 0.007	1.000
pair	0	0.707 ± 0.024	0.707 ± 0.024	1.000
linear CNOT	3	0.675 ± 0.034	0.667 ± 0.032	0.992 ± 0.002
kit ZZ map	24	0.440 ± 0.104	0.450 ± 0.114	0.870 ± 0.126

CNOT chain $\to$ Rot(β) $\to$ weighted Z readout, 15 gates and 18 noise locations. With $\alpha = 0$ it reduces to MOCO with a CNOT-chain feature map, whose last layer has a convex global optimum; we use that optimum as a warm start for Adam. The only hypothesis stated before the run, that the warm start beats a random initialization, holds: AUPRC 0.667 ± 0.010 against 0.635 ± 0.018, with half the spread over seeds (Table 1). Training inside the noise model at $p = 0.05$ brings nothing (0.666 ± 0.009). R2 stays below MOCO and costs 469,717 executions.

5.6 Ablations

Table 3 changes only the feature map. Three CNOTs already lower AUPRC and break the invariance; a second rotation per qubit (pair) adds nothing, which points to the information in four principal components as the ceiling. With the kit's ZZ encoding, even the globally optimal readout stays at 0.440 ± 0.104: the encoding, not the training, limits the baseline.

5.7 Adaptive shot triage

On hardware a transaction far from the threshold does not need 1,024 shots. One shot reads all four qubits and gives $h = \sum_q w_q z_q \in [-1, 1]$ with mean $\langle H \rangle_p$. We measure in batches of b shots and stop as soon as an anytime-valid confidence interval for $\langle H \rangle_p$ excludes the threshold $t_h = 1 - 2t_p$, with a cap $S_{\max}$. Three intervals were implemented: Hoeffding's [6] and the empirical Bernstein bound of Maurer and Pontil [7], both with a union bound $\alpha_k = 6\alpha/(\pi^2 k^2)$ over batches, and the predictable plug-in empirical-Bernstein confidence sequence of Waudby-Smith and Ramdas [8]. In all three cases the probability that the interval misses the mean at any batch is at most α, so the probability that a transaction is stopped early with a decision different from that of its exact score is at most α. In coverage tests on 10,000 simulated sequences all three stayed below α, whereas a fixed-sample interval rechecked at every batch exceeded it (`tests/test_adaptive.py`).

Selection on validation. We chose the encoding scale q (Section 5.9), the interval, b, α and $S_{\max}$ on validation at $p = 0.05$ (270 MOCO configurations, 5 seeds): highest validation F1, ties within 0.005 broken by fewer shots. The winner is the kit encoding with the plug-in interval, $b = 16$, $\alpha = 0.05$, $S_{\max} = 4096$ (validation F1 0.788 at 208.4 mean shots). The interval is the main lever: at tied F1, the Maurer–Pontil interval needs 991.7 shots and Hoeffding 917.9.

Test results. At $p = 0.05$ the triage uses 208.5 ± 46.8 shots per transaction instead of 1,024 (-80%) at the same F1 (0.801 ± 0.018 against 0.797 ± 0.019 with 1,024 fixed shots), with zero wrong early decisions in every MOCO run. At the same fixed budget for everyone (209 shots) F1 drops to 0.702 ± 0.060. Frauds, which lie closer to the threshold, use more shots than legitimate transactions (739.5 against 207.6). The mean shots grow like λ^{-4}: 161.0 $\to$ 208.5 $\to$ 579.6 from $p = 0$ to 0.05 and 0.2, ratios 1.30 and 3.60 against $\lambda^{-4} = 1.32$ and 3.46. AUPRC computed from the scores at stopping time drops from 0.716 to 0.700, because the triage deliberately leaves the far legitimate tail coarsely ranked. The baseline cannot be triaged under noise: at $p = 0.05$, 99.7% of its transactions reach the cap and F1 stays at 0.091 ± 0.072.

Shots, not executions. Each batch is one execution, so the triage costs 13.0 executions per transaction at $p = 0.05$ instead of one; the execution counts of Table 2 are unchanged. A `qml.Tracker` run on the kit's noisy device (5 rows) counted 79 executions and 1,264 shots, equal to the ledger formula $\sum_i K_i$ and $b \sum_i K_i$.

Confidence sweep. On validation only, with the selected configuration fixed, lowering the confidence $1 - \alpha$ from 0.999 to 0.51 reduces the mean shots from 404.7 to 94.8 at $p = 0.05$ (4.3×); F1 stays within 0.01 of its value at $\alpha = 0.05$ down to a confidence of 0.7 and loses about 0.02 at 0.51, and the rate of wrong early decisions never exceeds 0.0002 (Figure 5). $\alpha = 0.05$ was fixed before the sweep and is not changed.

5.8 Noise meter

Because s_0 is computable classically and $\mathbb{E}[\hat{s}_p] - \frac{1}{2} = \lambda^2(s_0 - \frac{1}{2})$ exactly, the least-squares slope of $(\hat{s}_p - \frac{1}{2})$ on $(s_0 - \frac{1}{2})$ over N_{cal} unlabeled calibration transactions is an unbiased estimate of λ^2, and $\hat{p} = \frac{3}{4}(1 - \sqrt{\hat{\lambda}^2})$.

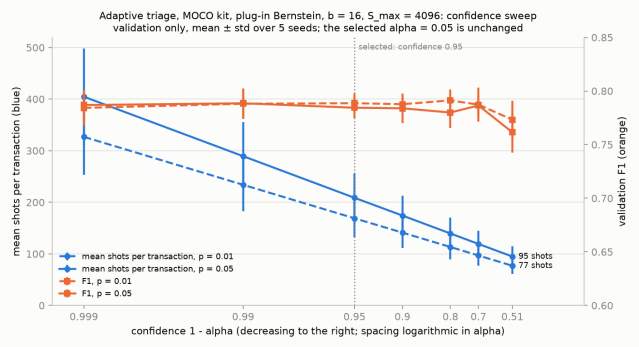


Figure 5: Confidence sweep of the adaptive triage on validation (5 seeds). Source: `1_confidence_sweep.png`, from `confidence_sweep.csv`.

With $S = 1024$ shots per calibration transaction, the relative RMSE of $\hat{p}$ for $N_{\text{cal}} = 50/200/1000$ is 47%/26%/11% at $p = 0.01$, 11%/5.3%/2.4% at $p = 0.05$ and 3.7%/1.8%/0.8% at $p = 0.2$, falling roughly as $1/\sqrt{N_{\text{cal}}S}$. With $t_{\hat{p}} = g_{\hat{p}}(t_0)$, test F1 stays within 0.0003 of the F1 of the threshold calibrated on the full noisy validation set at every N_{cal}. Threshold calibration thus costs 1,000 executions instead of 56,962. The rule fixed before the run (relative error at most 10% at every p) is not met at $p = 0.01$, where the error is 11%, without effect on F1.

5.9 Robust encoding (unselected comparator)

The kit's angle scaling is fixed by the full range of each principal component, which outlying frauds set. A robust variant fits the scale on the $[q, 100-q]$ percentiles of the training subset and clips to $[0, \pi]$. On validation it cuts the triage cost further, to 91.6 shots at $q = 0.5\%$ and 33.8 at $q = 5\%$, but lowers the validation F1 (0.778 and 0.732 against 0.788), so the selection rule kept the kit encoding. Carried to test as a comparator only, MOCO with $q = 0.5\%$ reaches AUPRC 0.738 ± 0.006 at every p, and the logistic regression on the same robust inputs 0.731 ± 0.004 (Table 1); the higher test AUPRC does not change a choice made on validation.

6 Limitations

- **Classically simulable.** The final state is a product state; scores are closed-form functions of the inputs. As every four-qubit circuit in this challenge, MOCO is simulable, and we claim no quantum advantage [9, 10].
- **Equivalent to a logistic regression** on $(\sin x_q, \cos x_q)$. MOCO matches, and does not beat, the logistic regression on the same inputs, on the official test set and on all 492 frauds.
- **Depolarizing noise only.** The theorem relies on the unital, isotropic channel of the official model; amplitude damping or readout errors would add bias terms and need separate analysis.
- **Four PCA inputs.** The precision-recall ceiling comes from the information in four principal components: at 80% recall the test precision is 0.817 ± 0.009, at 90% only 0.072 ± 0.032.
- **98 test frauds.** Test metrics move by whole frauds; we therefore add the cross-validation on all 492.
- **Triage cuts shots, not executions**, and its AUPRC is lower than with exact scores.

7 Related work

Data re-uploading and encoding. Re-uploading circuits [11] and the analysis of encodings as Fourier series [12] show that the encoding fixes the function class of a quantum model; MOCO is the extreme case of a single upload per qubit, whose class is linear in $(\sin x, \cos x)$. The ZZ feature map [13] used by the baseline is the opposite choice. *Classical surrogates and shadows.* Classical surrogates of quantum models [14] and classical shadows [15] learn from data measured once; MOCO does the same and compiles the result back into the circuit. *Simulability and trainability.* Barren plateaus [16], noise-induced plateaus [4] and initialization strategies [17] motivate our convex training and the warm start of R2; the observation that provably trainable models are often classically simulable [10] applies to MOCO. *Benchmarking.* Bowles et al. [9] argue that quantum models must be compared with classical baselines on the same inputs; we follow this throughout.

8 Conclusion

MOCO turns the training of a variational classifier into a convex fit on data measured once and compiles the optimum back into an 8-gate product circuit. Under the official depolarizing model its ranking, and with the analytically transferred threshold every threshold metric, are invariant at every noise level: test AUPRC 0.716 ± 0.007 against the baseline's drop to 0.405 ± 0.177, for 6,490 training executions instead of 2,859,360. Noise remains a cost on hardware, a shot tax of λ^{-4}, which adaptive triage and a noise meter reduce. The model matches a logistic regression on the same inputs; the gain is robustness, stability and cost. Next steps are hardware noise beyond depolarizing channels and more qubits, which cost nothing in robustness for product circuits.

Reproducibility. `make task2-test` runs the protocol, physics and interval tests; `make task2-experiments` reruns every experiment with fixed

seeds; `make task2-light` regenerates all tables, figures and slides from the CSVs.

References

[1] M. Cerezo, Andrew Arrasmith, Ryan Babbush, Simon C. Benjamin, Suguru Endo, Keisuke Fujii, Jarrod R. McClean, Kosuke Mitarai, Xiao Yuan, Lukasz Cincio, and Patrick J. Coles. Variational quantum algorithms. *Nature Reviews Physics*, 3: 625–644, 2021. doi: 10.1038/s42254-021-00348-9.

[2] Maria Schuld, Ville Bergholm, Christian Gogolin, Josh Izaac, and Nathan Killoran. Evaluating analytic gradients on quantum hardware. *Physical Review A*, 99:032331, 2019. doi: 10.1103/PhysRevA.99.032331.

[3] Dinuţ, Constantinescu, and Alexandrescu. Comparative analysis and noise robustness study of quantum kernel methods and variational quantum classifiers for financial fraud detection. *Electronics*, 15 (11):2489, 2026. doi: 10.3390/electronics15112489.

[4] Samson Wang, Enrico Fontana, M. Cerezo, Kunal Sharma, Akira Sone, Lukasz Cincio, and Patrick J. Coles. Noise-induced barren plateaus in variational quantum algorithms. *Nature Communications*, 12: 6961, 2021. doi: 10.1038/s41467-021-27045-6.

[5] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. Calibrating probability with undersampling for unbalanced classification. In *2015 IEEE Symposium Series on Computational Intelligence*, pages 159–166, 2015. doi: 10.1109/SSCI.2015.33.

[6] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58:13–30, 1963. doi: 10.1080/01621459.1963.10500830.

[7] Andreas Maurer and Massimiliano Pontil. Empirical Bernstein bounds and sample variance penalization. arXiv:0907.3740, 2009.

[8] Ian Waudby-Smith and Aaditya Ramdas. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 86:1–27, 2024. doi: 10.1093/jrsssb/qkad009.

[9] Joseph Bowles, Shahnawaz Ahmed, and Maria Schuld. Better than classical? The subtle art of benchmarking quantum machine learning models. arXiv:2403.07059, 2024.

[10] M. Cerezo, Martin Larocca, Diego García-Martín, N. L. Diaz, Paolo Braccia, Enrico Fontana, et al. Does provable absence of barren plateaus imply classical simulability? arXiv:2312.09121, 2023.

[11] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre. Data re-uploading for a universal quantum classifier. *Quantum*, 4:226, 2020. doi: 10.22331/q-2020-02-06-226.

[12] Maria Schuld, Ryan Sweke, and Johannes Jakob Meyer. Effect of data encoding on the expressive power of variational quantum-machine-learning models. *Physical Review A*, 103:032430, 2021. doi: 10.1103/PhysRevA.103.032430.

[13] Vojtěch Havlíček, Antonio D. Córcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567:209–212, 2019. doi: 10.1038/s41586-019-0980-2.

[14] Franz J. Schreiber, Jens Eisert, and Johannes Jakob Meyer. Classical surrogates for quantum learning models. *Physical Review Letters*, 131:100803, 2023. doi: 10.1103/PhysRevLett.131.100803.

[15] Hsin-Yuan Huang, Richard Kueng, and John Preskill. Predicting many properties of a quantum system from very few measurements. *Nature Physics*, 16:1050–1057, 2020. doi: 10.1038/s41567-020-0932-7.

[16] Jarrod R. McClean, Sergio Boixo, Vadim N. Smelyanskiy, Ryan Babbush, and Hartmut Neven. Barren plateaus in quantum neural network training landscapes. *Nature Communications*, 9:4812, 2018. doi: 10.1038/s41467-018-07090-4.

[17] Edward Grant, Leonard Wossnig, Mateusz Ostaszewski, and Marcello Benedetti. An initialization strategy for addressing barren plateaus in parametrized quantum circuits. *Quantum*, 3:214, 2019. doi: 10.22331/q-2019-12-09-214.